

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 13 · LECTURE NOTES

Using Randomness Honestly

Seeds, sources, and claims you can support.

MODULE OUTCOMES

- Report a randomised result reproducibly.
- Explain how a weak random source voids a guarantee.
- Distinguish the kinds of random source and their uses.
- Measure a failure probability against its bound.
- State what a randomised algorithm honestly promises.

1 Reproducibility, and measuring the bound

- **An explicit seed, logged alongside the result.** A result you cannot reproduce is a result you cannot debug, and 'it worked yesterday' is not a report — which is obvious and is omitted constantly.
- **Multiple seeds, with the spread reported** — CSCE 642 Module 12's five-seed minimum and Module 09 §4's four chains are the same discipline in different settings, and the reason is identical: a single run of a stochastic process is an anecdote.
- **A named generator and library version.** Generators differ between library versions and between languages, so a seed alone does not determine the random stream — and a library upgrade silently changing your results is a genuinely confusing failure.
- **Per-component seeding rather than one global seed.** A single global generator couples unrelated components, so adding one extra draw in one place shifts every subsequent value everywhere — which makes bisecting a regression impossible.
- **And a record of what was *not* controlled** — thread scheduling, hash iteration order, floating-point reduction order, GPU kernel non-determinism. **Full determinism is usually unattainable, and the honest position is to claim reproducibility only for what you actually controlled and to list the rest.**

MEASURE THE FAILURE RATE, AND COMPARE IT AGAINST YOUR BOUND

You derived a bound in Module 02. Now run the algorithm enough times to estimate the actual failure rate on your inputs. This is the central activity of the course, and it is what the first project is built around.

And they will differ — the bound is worst-case over all inputs and the measurement is on yours, so the measured rate is usually far better, frequently by orders of magnitude (Module 05 §4's caveat, which is good news).

Report both. 'Bound 10^{-6} , measured 3×10^{-9} over 10^9 trials' is a far stronger statement than either figure alone — the bound establishes the guarantee and the measurement demonstrates the practice.

And if the measurement is worse than the bound, something is wrong — usually an independence assumption that does not actually hold (Module 02 §4's commonest error) **or a weak generator** (§2).

Which makes the comparison a test of your analysis as well as of your implementation, and that dual role is why it is worth the effort.

2 The source

Source	Use it for	Never use it for
Mersenne Twister, PCG, xoshiro	Simulation, randomised algorithms, sampling, shuffling for non-adversarial purposes.	Anything adversarial — these are predictable from a modest number of outputs.
The OS cryptographic source (urandom, getrandom, BCRYPTGenRandom)	Keys, nonces, session tokens, salts, and anything an adversary might try to predict.	Nothing — it is always safe, and the performance objection is obsolete on modern systems.
A fixed, logged seed	Reproducible experiments and tests.	Production, ever — a shipped fixed seed is a predictable system.
A time-based seed	Nothing.	Everything — see the note.
A hardware RNG instruction or device	Seeding a cryptographic generator.	Direct use without whitening or health testing, since the raw output may be biased.

The time-based seed row is not pedantry. Seeding from the clock has broken real cryptographic systems, because the clock's entropy as seen by an attacker who knows roughly when the key was generated is a handful of bits — which is exhaustively searchable. **It is a repeated class of incident rather than a theoretical concern**, and the fix is free.

A WEAK SOURCE VOIDS THE GUARANTEE RATHER THAN DEGRADING IT

Every bound in this course assumes the randomness is what you said it was — independent, uniform, and (for the adversary use) unknown to whoever is choosing the input.

And a weak source breaks the assumption completely rather than partially. A predictable generator against an adversary gives you a worst-case input, not a slightly worse average (Module 01 §1) — the adversary computes your choices and constructs the input that defeats them. **The guarantee does not degrade; it disappears.**

The documented failures are real and numerous: predictable session tokens allowing account takeover, cryptographic keys generated with insufficient entropy at first boot (which produced thousands of duplicate keys in deployed devices), and hash-flooding denial of service against unrandomised hash tables have all caused serious incidents.

So the test is cheap and worth doing: run your algorithm with a deliberately weak generator — a short-period one, or one seeded identically every time — and see whether the guarantee survives. And if the result is entirely unchanged, your randomness was not doing what you thought it was doing, which is itself worth discovering. This is the one experiment in the course that nobody runs.

3 The claim

- 'Expected $O(n \log n)$ time, with probability at least $1 - n^{-2}$ of not exceeding twice that; measured over 10^6 runs, the maximum observed was 1.3 times the mean.' The bound and the measurement together (§1).
- 'Failure probability at most 2^{-40} , assuming independent uniform bits from the operating system's source.' The assumption named explicitly, because that is where failures actually come from (§2).
- 'Estimates within 2% with 99% confidence from 1.5 KB of state; measured error 0.4% on our production data over a month.' A sketch's claim, with both sides (Module 05 §4).
- 'Results over ten seeds; interquartile mean 312, range 280 to 340; the seeds and the library version are in the appendix.' Which is CSCE 642 Module 12's reporting standard, and it transfers directly.
- And what you cannot honestly say: 'it is fast' or 'it works' — without the probability, without the randomness assumption, or without the seeds. Every honest claim above names the probability and the assumption: the first because it is the guarantee, and the second because it is what the guarantee rests on.

THE THREE QUESTIONS TO ASK OF ANY RANDOMISED RESULT

What is the failure probability, and is it *bounded* or merely *observed*? An unbounded empirical rate is a measurement rather than a guarantee — useful, and a different kind of claim.

What does the analysis assume about the randomness? Independence is the usual assumption and the usual place it fails (Module 02 §2), and an analysis that does not state its independence assumption has probably not checked it.

How many seeds, and is the spread shown? CSCE 642 Module 12's question, and it applies here identically — a randomised algorithm reported from one run is reported from an anecdote.

And the three are listed in order of how often they are omitted — the seed count is usually missing entirely, the independence assumption is usually unstated, and the failure probability is usually present, which means the reporting is weakest exactly where it is easiest to fix.

4 The semester, and the close

Course	Asked	And answered
CSCE 627	Can it be done at all?	Unconditionally, with proofs that do not expire (CSCE 627 Module 13 §1).
CSCE 637	What does it cost?	Mostly conditionally, and three barrier results explain why the central question is open (CSCE 637 Module 13 §1).

Course	Asked	And answered
CSCE 658 (this one)	Does a coin help?	Demonstrably and repeatedly in practice; whether it is ever <i>essential</i> for a complexity class is open (Module 12 §3).

And this is the course whose content you will use most. Hashing, sketches, sampling, randomised backoff, and the power of two choices are in essentially everything you will build — while CSCE 627 and CSCE 637 change how you think about problems rather than what you write. Being clear about which of the three is immediately practical is honest, and it is not a judgement about which is more valuable.

WHERE THIS COURSE LEAVES YOU

You can derive a failure bound using the right inequality from Module 02's ladder, state the independence assumption it needs, and measure whether the derivation actually held — which is the core competence and is what the projects assess.

You can design a sketch, a randomised data structure, or a rounding scheme and bound its error — which covers most of what randomness is actually used for in systems (Modules 04, 05, 08).

And you can recognise where randomness is *necessary* (Module 11's symmetry breaking and consensus) rather than merely convenient, and where it should be removed again (Module 12's four criteria).

The closing rule is the program's, unchanged across twenty-four courses: state what you measured, state what you assumed, and never claim more than you established. In this course it means naming the randomness assumption and the seed — because every single guarantee in these thirteen modules is conditional on a source being what you said it was, and that is the assumption nobody writes down.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Agarwal et al. — Deep RL at the Edge of the Statistical Precipice (free)	§1's reporting standard , developed for reinforcement learning and applicable unchanged here — with free tooling.
O'Neill — PCG, and the random number generator survey (free)	§2's generators compared , including the statistical tests and the predictability discussion.
Heninger et al. — Mining Your Ps and Qs (free)	§2's boot-entropy failure, measured in the wild — thousands of duplicate keys in deployed devices. The canonical example.
Brooker — the AWS architecture posts on jitter and timeouts (free)	Module 11 §1's practice, documented — and a model for how to report a randomised design decision.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Take every experiment from this course and make it reproducible:** explicit seeds, named generator, logged versions.
- 2 **Switch to a global seed** and demonstrate the coupling between components.
- 3 **Measure the failure rate of three algorithms** you implemented and compare against your bounds.
- 4 **Find one where the measurement is worse than the bound**, and diagnose it.
- 5 **Run one algorithm with a deliberately weak generator** and report whether the guarantee survived.
- 6 **Seed a generator from the clock** and estimate how many seeds an attacker would need to try.
- 7 **Write the honest claim** for each algorithm in your first project.
- 8 **Apply the three questions** to a published randomised result and report what was missing.
- 9 **Revisit what you wrote in Module 01's last exercise** and report what changed.
- 10 **Project 2 is now due.** Submit the problem, the deterministic baseline measured, the randomised algorithm with its derived and measured failure probability, ten seeds with the spread, the weak-generator sensitivity test, a derandomisation attempt or an argument against one, and the honest claim.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five requirements for a reproducible randomised experiment.
- 2 Why seed per component rather than globally?
- 3 Why report both the bound and the measured rate?
- 4 What does it mean if the measurement is worse than the bound?
- 5 Name five random sources and what each is and is not for.
- 6 Why is a time-based seed never acceptable?
- 7 Why does a weak source void rather than degrade a guarantee?
- 8 Give four honest claims and the two things you cannot say.
- 9 Give the three questions to ask of a randomised result, in order of omission.
- 10 In this course, what does 'state what you assumed' mean?